

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

C04 – ASSESSMENT TOOL 1

Comparative Analysis

Heap vs Sorted Array • Linear Probing vs Separate Chaining

NAME	Alex Maddy
REG. NO	192565060
COURSE NAME	Data Structure
COURSE CODE	CSA0307
DATE OF SUBMISSION	17 August 2026

*Submitted in partial fulfilment of the requirements of
the course CSA0307 – Data Structure*

Question 1: Heap vs Sorted Array for Priority-Based Insertion and Retrieval

A scheduling-type system that must repeatedly (a) insert new elements and (b) retrieve the highest-priority element is best analysed by comparing a Heap (typically a binary heap) with a Sorted Array as the underlying data structure.

a) Insertion Time

Heap: A new element is placed at the next free slot (end of the array-backed tree) and then "bubbled up" by comparing it with its parent, swapping until the heap property is restored. Since a binary heap has height $O(\log n)$, insertion takes $O(\log n)$ time.

Sorted Array: A new element must be placed at the correct position so the array stays sorted. Binary search can locate the position in $O(\log n)$, but all subsequent elements must be shifted right to make room, making the overall insertion cost $O(n)$ in the worst case.

b) Deletion Time (Retrieving the Highest-Priority Element)

Heap: The root (highest-priority element, e.g. maximum in a max-heap) is removed in $O(1)$ to identify, but the heap must then be re-organised: the last element is moved to the root and "sifted down" to restore the heap property, costing $O(\log n)$ overall.

Sorted Array: If the array is sorted so the highest-priority element is at the end (or beginning), removing it is $O(1)$. However, if it is at the beginning, all remaining elements must shift left, costing $O(n)$.

c) Efficiency in Real-Time Systems

- Heap: Both insertion and deletion run in $O(\log n)$, giving predictable, bounded, and consistently low latency as the number of elements (n) grows — important for meeting real-time deadlines.
- Sorted Array: Insertion is $O(n)$ in the worst case, which becomes a bottleneck as n grows; performance degrades and is less predictable under frequent insertions, even though best-case deletion is $O(1)$.
- Heaps offer balanced, scalable performance for both operations, whereas sorted arrays only excel at one operation (deletion) at the cost of the other (insertion).

Operation	Heap	Sorted Array
Insertion	$O(\log n)$	$O(n)$ (due to shifting)
Deletion (highest priority)	$O(\log n)$	$O(1)$ best case, $O(n)$ if shifting needed
Search for arbitrary element	$O(n)$	$O(\log n)$
Space overhead	Low (implicit array-based tree)	Low (plain array)
Predictability under load	Consistent, bounded	Degrades as n increases

Recommendation for Scheduling Systems

A Heap (typically implemented as a Priority Queue) is the better choice for scheduling systems. Scheduling systems constantly add new tasks/processes and repeatedly extract the highest-priority one — a workload that maps directly onto a heap's $O(\log n)$ insertion and $O(\log n)$ extraction. This gives balanced, predictable performance regardless of how many tasks are queued, which is essential for CPU schedulers, event-driven simulations, and real-time task managers. A sorted array only performs well when deletions dominate and insertions are rare, which is not the case in most scheduling workloads.

Question 2: Linear Probing vs Separate Chaining for Collision Handling

Hashing relies on a collision-resolution strategy whenever two keys map to the same index. Linear Probing (an open-addressing technique) and Separate Chaining (a closed-addressing technique) are the two most common approaches, compared below.

a) Clustering Effect

Linear Probing: When a collision occurs, the algorithm searches sequentially for the next free slot. This causes primary clustering — long runs of occupied slots form, and once a cluster forms it tends to grow faster, increasing the average probe length for future insertions and lookups.

Separate Chaining: Each bucket holds an independent linked list (or similar structure) of all keys hashing to that index. Since colliding keys are simply appended to a list rather than probing other slots, there is no clustering effect on the underlying table at all.

b) Memory Usage

Linear Probing: Uses only the fixed-size array itself — no extra pointers or node overhead — but the table must always keep free slots (load factor comfortably below 1) to avoid excessive probing, which means some space is reserved and "wasted" as headroom.

Separate Chaining: Requires additional memory for the linked-list nodes and pointers, so per-element overhead is higher. However, it can gracefully handle a load factor greater than 1 without failing, since buckets simply grow longer.

c) Performance Under High Load Factor

- Linear Probing: Performance degrades sharply as the load factor approaches 1 — clusters merge, probe sequences become very long, and insertion/search can approach $O(n)$ in the worst case. It cannot exceed a load factor of 1 (table becomes full).
- Separate Chaining: Degrades more gracefully — average search/insert cost grows linearly with the load factor ($O(1 + \alpha)$, where α is the load factor), since only the affected bucket's chain lengthens rather than the whole table being disturbed.
- Separate chaining remains usable even when $\alpha > 1$, whereas linear probing requires resizing/rehashing well before the table fills up to maintain acceptable performance.

Criteria	Linear Probing	Separate Chaining
Clustering	Suffers from primary clustering	No clustering (independent buckets)
Memory overhead	Low (no pointers, but needs free slots)	Higher (extra pointers/nodes per entry)
Behaviour at high load factor	Degrades sharply; capped at load factor 1	Degrades gracefully; handles load factor > 1
Cache performance	Better (array locality)	Weaker (pointer chasing, scattered nodes)
Implementation complexity	Simple array logic, needs deletion care (tombstones)	Simple list logic, straightforward deletion

Recommendation for Large-Scale Applications

Separate Chaining is generally more suitable for large-scale applications. Large systems experience unpredictable and often high load factors as data grows, and separate chaining degrades gracefully in that scenario, is simpler to resize dynamically, and avoids the clustering problem that cripples linear probing under heavy load. Linear probing can still be attractive when memory is tightly constrained and cache-friendly array access is critical (e.g. low-level or embedded systems) with a well-managed, low load factor — but for general large-scale, growth-prone applications, separate chaining offers more robust and predictable performance.

Overall Conclusion

Both comparisons illustrate the same underlying principle in data structure design: the right choice depends on which operation dominates the workload and how the system needs to behave as it scales.

Summary
Priority-based systems → choose the Heap. For workloads that repeatedly insert elements and extract the highest-priority one, a Heap keeps both operations at $O(\log n)$, giving scheduling systems consistent, predictable performance regardless of how large the task queue grows.
Large, growth-prone systems → choose Separate Chaining. For hash-based storage that must tolerate unpredictable and high load factors, Separate Chaining degrades gracefully and avoids the clustering that limits Linear Probing, making it the safer default for large-scale applications.

Key Takeaway

In both cases, the structure that offers balanced, predictable performance as data volume grows (Heap for priority queues, Separate Chaining for hash tables) is preferable to one that is only efficient for a single operation or under low-load conditions. Real-world scheduling and large-scale systems must be designed for growth, not just for the average case.

— End of Document —